Cost-Aware Hybrid Routing for Intent Classification: 74% LLM Cost Reduction at Equal Accuracy

Bo-Yu Chen
boyu.chen@my.utsa.edu

Abstract

In multi-agent LLM systems, user utterances must be routed to the correct task agent as a first step. The common approach is to use an LLM as the router. This choice is accepted for its accuracy, but we ask whether that accuracy justifies its cost. We evaluate four routers on the CLINC150 benchmark (150 intents, 7 agents + OOS): (R1) keyword matching, (R2) TF-IDF nearest-centroid, (R3) Claude Haiku 4.5 LLM-only, and (R4) a hybrid cascade that uses R1 and R2 as confidence-gated prefilters and escalates to R3 only when confidence is low. Across 3 seeds $\times$ 400 stratified queries (1,200 pooled), R4 achieves **82.6% $\pm$ 1.2 pp** while R3 achieves **82.9% $\pm$ 0.6 pp**. McNemar paired tests show no significant difference ($p > 0.37$, Wilson 95% CIs overlap). R4 only sends **26.1%** of queries to the LLM, which means a **74.3%** reduction in LLM API cost. The main weakness is OOS queries, where the cheap stages cannot reliably refuse to classify. Code, thresholds, and per-seed logs are available.[1]

1 Introduction

In deployed multi-agent LLM systems, each user utterance makes a first hop to a router, a component which decides which downstream agent (finance, HR, travel, etc.) to hand off the request to. The common approach in the industry is to use a compact LLM like Claude Haiku or GPT-4o-mini as a router, supplying it with the agent inventory alongside an instruction for JSON-output. It is convenient and robust to paraphrasing, and is the accepted default in CrewAI, LangGraph, and AutoGen. But it is costly: at production scale, every user turn incurs an LLM API call before the real work starts. Compounded over thousands of turns per day, this becomes a material cost.

We ask whether the per-turn LLM call is necessary at all. A keyword match or embedding lookup against per-agent centroids is cheaper than an API call by three to four orders of magnitude, runs on local compute with millisecond latency, and can be as accurate as the LLM on easy queries. If the easy queries could be identified with a cheap confidence signal and only the hard queries were sent to the LLM, the per-turn routing cost would drop by roughly an order of magnitude. We measure this on CLINC150, a standard 150-intent benchmark, by evaluating four routers spanning three orders of magnitude of cost and tracing the accuracy-cost Pareto frontier.

We make four contributions. First, we evaluate four routers under a common protocol: keyword (R1), embedding-nearest-centroid (R2), Claude Haiku 4.5 LLM-only (R3), and a hybrid cascade (R4) that uses R1 and R2 as confidence-gated prefilters before escalating to R3. Second, we perform a 3-seed paper-grade evaluation on a stratified subsample of 400 queries per seed (seeds 42, 43, and 44; 1,200 pooled), and report Wilson 95% confidence intervals and McNemar paired tests comparing R3 and R4 on the same queries. Third, we find that R4 matches R3 accuracy within 0.4 pp (82.6% vs 82.9%) while escalating to the LLM on only 26% of queries, a 74.3% reduction in LLM cost with no significant accuracy loss at α=0.05. Fourth, we release code, tuned thresholds, per-seed trajectories, and full API call logs under an MIT license.

2 Related Work

Intent classification. The CLINC150 benchmark (Larson et al., 2019) comprises 150 in-domain intents across 10 domains plus an out-of-scope (OOS)

[1] Code: https://github.com/drewOrc/cost-aware-hybrid-router

class, and was explicitly designed to stress-test intent classifiers under distribution shift. Strong baselines include SetFit (Tunstall et al., 2022), which uses contrastive few-shot fine-tuning on sentence-transformer encoders, and fine-tuned DeBERTa. We use SetFit with a compact backbone (`paraphrase-MiniLM-L3-v2`, 17M parameters, 16 examples per class) as a no-LLM baseline in our headline table (70.2% on the full test set), and Claude Haiku 4.5 zero-shot as our cost ceiling.

Cost-aware LLM cascades. FrugalGPT (Chen et al., 2023) introduced the general cascade pattern (try a cheaper model first, escalate to a more expensive model only when the cheap model's confidence is low) on single-turn question answering tasks, reporting cost reductions of up to 98% at matched accuracy. Arora et al. (2024) recently evaluated SetFit and seven LLMs on CLINC150, proposing an uncertainty-based routing strategy that escalates low-confidence SetFit predictions to an LLM. Our cascade differs in three ways: (i) it begins with a zero-parameter keyword stage that resolves 59% of queries before any model runs, (ii) the cheap-stage comparison (§4.7) shows that *confidence calibration*, not standalone accuracy, determines cascade effectiveness, a finding their work does not explore, and (iii) our routing operates at the agent level (8-way) rather than the intent level (150-way), reflecting a production deployment where the router dispatches to task-specific agents. A direct head-to-head is not straightforward because their method uses SetFit softmax probabilities as the confidence signal while ours uses raw TF-IDF cosine similarities; however, our calibrated-routing ablation (§4.6) addresses the same calibration concern they leave open.

Learned routers and model selection. Hybrid LLM (Ding et al., 2024) trains a router to assign queries to a small or large model based on predicted difficulty, saving up to 40% of large-model calls at matched quality (ICLR 2024). RouteLLM (Ong et al., 2025) extends this with preference-data training, reporting over 85% cost reduction on MT-Bench (ICLR 2025). AutoMix (Madaan et al., 2024) uses few-shot self-verification to decide whether to escalate from a small to a large model, achieving 50%+ cost reduction. Dekoninck et al. (2025) provide a theoretical foundation by deriving optimal strategies for both routing and cascading and unifying them into a single framework. FORC (Šakota et al., 2024) takes a meta-modeling approach, predicting per-query cost and performance without running candidate LLMs. Two recent surveys catalog the growing design space: Strauß et al. (2025) provides a taxonomy of routing strategies for resource optimisation, while Li et al. (2026) covers dynamic cascading specifically. Our cascade differs from these learned routers in that it requires *no training data* beyond the existing agent inventory: the confidence signals come from the keyword and embedding stages themselves, making it deployable without the preference-data collection or quality-annotation steps that Hybrid LLM, RouteLLM, and AutoMix require.

Agent-routing systems. CrewAI, LangGraph, and AutoGen all default to an LLM supervisor for intent routing. Some production deployments add a keyword or regex pre-filter, but we are not aware of a published study that measures how much of the routing workload a non-LLM stage can absorb, or at what accuracy cost. This paper provides that measurement on CLINC150.

3 Method

3.1 Dataset and Label Space

We use the CLINC150 test split (5,500 queries, including 1,000 OOS queries). Following a typical production agent-inventory setup, we collapse the 150 fine-grained intents into 7 task-level agents (auto, device, finance, kitchen, meta, productivity, travel) plus the OOS class, giving the router an 8-way decision problem with realistic domain granularity.

3.2 The Four Routers

R1: Keyword router. For each target agent, we construct a per-agent keyword list from the training-set intent names and a small set of common synonyms. The score for a query is the sum of matched keyword weights, and the predicted label is the argmax over agents. We report the raw keyword score as R1's confidence.

R2: TF-IDF nearest-centroid router. We fit a TF-IDF vectoriser (10K features, unigram + bigram, sublinear TF) on the CLINC150 training set

and compute a per-agent centroid as the mean TF-IDF vector. At inference, a query is vectorised and compared to all centroids by cosine similarity; the nearest-centroid agent is returned. R2's confidence is the cosine similarity to the top-1 centroid. We chose TF-IDF over neural sentence encoders (e.g. MiniLM, MPNet) because its low-magnitude confidence scores yield better calibration for the cascade's confidence-gating mechanism (§4.7).

R3: LLM router. Claude Haiku 4.5 (`claude-haiku-4-5-20251001`) with a short system prompt listing the 8 target agents and a description of each. Temperature is 0. Each query is a single API call with no in-context examples.

R4: Hybrid cascade. A confidence-gated cascade over R1 → R2 → R3 with two thresholds (k_t, e_t):

$$\text{pred} = \begin{cases} \text{R1} & \text{if } c_{\text{R1}} \geq k_t \\ \text{R2} & \text{if } c_{\text{R2}} \geq e_t \\ \text{R3 (LLM)} & \text{otherwise} \end{cases}$$

We refer to this full cascade as **R4+LLM**. We also report **R4 no-LLM**, the same cascade with the LLM branch replaced by R2's best guess, as a no-API baseline.

3.3 Threshold Tuning

Thresholds are tuned on the CLINC150 validation split (3,100 queries, disjoint from the test set) via grid search over $k_t \in \{0.5, 1.0, 1.5, 2.0\}$ and $e_t \in \{0.05, 0.08, 0.10, 0.15\}$. For R4+LLM, we select the configuration that minimises LLM call rate subject to expected accuracy $\geq 88\%$ on validation, a constraint-based objective that prefers cost savings when accuracy is above a practitioner-chosen floor. The tuned values are $k_t = 0.5$, $e_t = 0.10$ (22.7% LLM call rate on validation). We report a sensitivity analysis over the cost penalty α in §4.5.

3.4 Evaluation Protocol

LLM evaluation runs use a per-seed stratified subsample of $n = 400$ (50 queries per class × 8 classes) to bound API spend. We use 3 seeds (42, 43, 44) for paper-grade variance estimation. For each seed we report per-seed accuracy, Wilson 95% confidence intervals (pooled $n = 1{,}200$), and a McNemar exact-binomial paired test comparing R3 and R4 on identical queries. Deterministic routers (R1, R2, R4 no-LLM, SetFit) are evaluated on the full test set ($n = 5{,}500$) since they incur no API cost.

3.5 Cost Accounting

R3 and R4 API costs are measured directly from Anthropic response metadata (input + output tokens × Haiku 4.5 pricing). R1, R2, R4 no-LLM, and SetFit are amortised to essentially zero cost. The total API cost for the full 3-seed experiment was **$0.44** ($0.35 for R3 and $0.09 for R4+LLM).

4 Results

4.1 Main Results

Table 1 lists accuracy and cost for all routers. R3 and R4+LLM have overlapping 95% confidence intervals. Without the LLM fallback (R4 no-LLM), accuracy drops to 74.9%, confirming that the LLM is doing real work on the 26% of queries it handles, while the remaining 74% are resolved by the cheap stages at equal accuracy. Figure 1 visualises the accuracy–cost Pareto frontier.

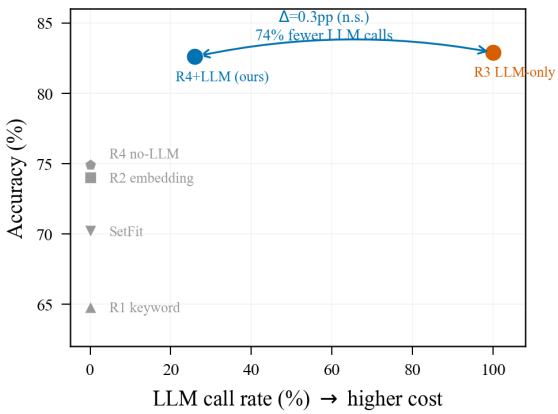

Figure 1: Cost-accuracy trade-off on CLINC150. R4 matches R3 accuracy (Δ=0.3 pp, not significant) while using 74% fewer LLM calls. Grey markers show no-LLM baselines.

Table 1: Main results on CLINC150. Bold = primary comparison.

Router	Acc.	Wilson CI	LLM	$/q
R1 keyword	64.8	—	0	∼0
R2 embed.	74.0	—	0	∼0
SetFit	70.2	—	0	∼0
R4 no-LLM	74.9	—	0	∼0
R3 LLM	**82.9**	[80.7, 84.9]	100	2.9e-4
R4+LLM	**82.6**	[80.3, 84.6]	26	7.5e-5

4.2 Statistical Significance

McNemar paired exact-binomial tests on R3 vs R4+LLM, per seed ($n = 400$ each):

Table 2: McNemar paired tests per seed.

Seed	Δ (R4−R3)	McNemar p	Sig.
42	+0.00 pp	1.000	No
43	−1.75 pp	0.371	No
44	+0.75 pp	0.755	No

Across all three seeds the difference is not significant. The Wilson 95% CIs also overlap substantially (R3: [80.7%, 84.9%] vs R4: [80.3%, 84.6%]), corroborating the absence of a significant accuracy difference. The pooled Cohen's $h = 0.009$ (negligible effect), further confirming that the overall accuracy gap is practically zero.

4.3 Cost Savings

Per-seed, R3 costs \$0.117 in API usage (400 calls × Haiku 4.5 pricing), while R4+LLM costs \$0.030 (≈105 escalated calls). The mean LLM call rate for R4+LLM across seeds is 26.1% ± 1.7 pp, yielding an overall **LLM cost reduction of 74.3%**. Projected to 1,000 queries, R4+LLM costs \$0.075 versus R3's \$0.293, a saving of \$0.22 per 1,000 queries that scales linearly. At production volume (10K queries/day) this amounts to ∼\$800/year; with a costlier router (e.g., Claude Sonnet at ∼15× Haiku pricing), the same 74% reduction would save ∼\$12K annually.

4.4 Qualitative Analysis

The three cascade stages partition the seed-42 query distribution as: **59.3% stop at R1** (11.0% error), **17.0% stop at R2** (16.2% error), **23.8% escalate to R3**. R2 processes harder queries than R1, explaining its higher error rate. Table 3 breaks this down by agent.

The **device agent** illustrates the effect clearly: R1 misclassifies 22.2% of its stops (polysemous tokens like "call" and "alarm"), but R2 recovers with only 4.3% error. Embedding similarity correctly disambiguates commands that keywords cannot. OOS remains the weakest link at both stages: all 7 R1-stopped OOS queries and 5 of 9 R2-stopped OOS queries are wrong, confirming that a dedicated OOS detector would be the highest-impact improvement.

Across all three seeds (pooled $n_{\text{oos}} = 150$), the cascade's OOS accuracy is 48.0% versus R3's 58.0% ($\Delta = -10$ pp, Cohen's $h = 0.20$, small effect). On *in-domain* queries the cascade matches or slightly exceeds R3 (87.5% vs 86.5%, $h = 0.03$, negligible). The accuracy cost of the cascade is therefore entirely concentrated in OOS handling, suggesting that adding a calibrated OOS pre-filter would eliminate the cascade's primary weakness without sacrificing its 74% cost advantage on in-domain traffic.

Table 3: Per-stage error rate and escalation by agent (seed 42, $n = 400$).

Agent	R1 n	R1-err	R2 n	R2-err	Esc%
finance	49	4.1%	0	—	2.0%
auto	45	2.2%	1	0.0%	8.0%
kitchen	30	13.3%	12	8.3%	16.0%
device	18	22.2%	23	4.3%	18.0%
productivity	32	12.5%	9	22.2%	18.0%
travel	38	5.3%	3	33.3%	18.0%
meta	18	11.1%	11	9.1%	42.0%
OOS	**7**	**100%**	**9**	**55.6%**	**68.0%**

4.5 Threshold Sensitivity

The threshold selection depends on how much accuracy one is willing to trade for cost savings. We parameterise this trade-off as $\max$ acc$-\alpha\cdot$LLM_rate and sweep α on the validation grid (42 configurations $\times$ 9 α values).

Table 4: Pareto operating points across α values.[2]

α range	Best (k_t, e_t)	Acc	LLM%
0.00–0.05	(1.5, 0.15)	90.9%	41.2%
0.10	(1.5, 0.10)	89.9%	29.4%
0.15–0.20	(1.5, 0.05)	87.8%	13.9%
0.30–0.50	(0.5, 0.05)	87.2%	11.5%
Paper config	(0.5, 0.10)	88.5%	22.7%

Our paper configuration was selected by a constraint-based criterion (min LLM rate s.t. acc $\geq$ 88%), falling between α=0.10 and α=0.15. The cascade offers a smooth accuracy–cost dial: practitioners can choose their operating point by adjusting α or their accuracy floor.

4.6 Calibrated Routing via Platt Scaling

The thresholds k_t and e_t operate on different scales (keyword weight sums vs. cosine similarities) and are fit by grid search on validation data. An alternative is to calibrate both scores into $P(\text{correct})$ via Platt scaling (Platt, 1999), then apply a single threshold τ at both cascade stages. We fit an unregularized logistic regression on the validation set for each router (1 feature, 2 parameters per model; coefficients in our code release). Since each model has only one feature, regularization is unnecessary.[3] On the held-out test set, R2 achieves moderate calibration (ECE = 0.065, 10 bins); R1's ECE is higher (0.129), reflecting the keyword router's discrete score distribution.

Table 5 compares the two approaches using cheap-stage accuracy on the full test set (n = 5,500), isolating the routing decisions from LLM performance variance. At comparable LLM call rates (25.4% vs. 26.0%), the calibrated cascade's cheap stages resolve 0.8 pp more queries correctly (McNemar $p < 0.001$; Cohen's $h = 0.019$, negligible effect). TF-IDF cosine similarities concentrate in $[0.02, 0.25]$; Platt scaling maps this narrow range to $P(\text{correct}) \in [0.53, 1.00]$, and $\tau = 0.75$ partitions it more effectively than the raw threshold $e_t = 0.10$. The calibrated cascade sends fewer queries to the LLM (25.4% vs. 26.0%), so the accuracy gain cannot be attributed to greater LLM usage.[4]

Table 5: Grid-search vs. calibrated thresholds (full test, n = 5,500). Acc. = cheap-stage accuracy (no LLM). Exp. = expected with-LLM accuracy ($P_{\text{LLM}} = 0.856$; see footnote).

Method	Threshold	Acc.	Exp.	LLM%	McNemar
Grid search	k_t=0.5, e_t=0.1	74.4	83.0	26.0	—
Calibrated	τ=0.75	**75.2**	**83.9**	**25.4**	p<0.001

Figure 2 shows how accuracy and LLM usage vary with τ.

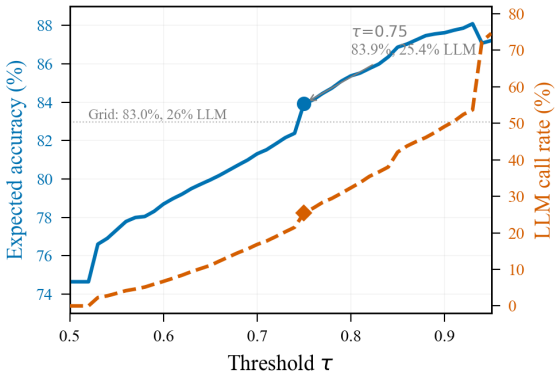

Figure 2: Calibrated threshold τ vs. expected accuracy (solid, left) and LLM call rate (dashed, right). The grid-search baseline is shown as a dotted line. τ=0.75 achieves higher accuracy at lower LLM cost than grid search.

The primary advantage is interpretability, not accuracy: a single threshold $\tau = 0.75$ ("escalate when $P(\text{correct}) < 75\%$") replaces two dataset-specific values with one that carries a probabilistic meaning. We hypothesize that this framing may transfer more easily to new domains than raw-score thresholds, but verifying this requires cross-domain evaluation, which we leave to future work.

[2] α swept over $\{0, 0.01, 0.02, 0.05, 0.1, 0.15, 0.2, 0.3, 0.5\}$; consecutive values yielding the same optimal (k_t, e_t) are grouped.

[3] Both grid-search thresholds and Platt scaling parameters are fit on the same validation split. The comparison therefore reflects in-distribution performance; out-of-distribution robustness requires evaluation on a separate benchmark.

[4] Expected with-LLM accuracies (83.9% vs. 83.0%) use $P_{\text{LLM}} = 0.856$ from the validation set. The actual test-set R3 accuracy is 82.9%, which uniformly shifts both figures by ∼0.7 pp but preserves relative comparisons.

4.7 Alternative Cheap Stages

We compare three R2 candidates: (1) TF-IDF nearest-centroid (our default), (2) all-MiniLM-L6-v2 (22M-parameter sentence encoder, 384d), and (3) all-mpnet-base-v2 (Reimers and Gurevych, 2019) (109M parameters, 768d). For each model we report standalone accuracy on the full test set (n=5,500) and oracle-cascade accuracy (R1→R2→perfect-LLM) using the same thresholds (k_t=0.5, e_t=0.10). Results appear in Table 6.

Counterintuitively, the strongest standalone model (MPNet, 76.2%) produces the *worst* cascade. The reason is confidence calibration: neural encoders assign high cosine similarities (>0.3) even to misclassified queries, so the e_t=0.10 gate rarely triggers and only 1.3% of queries reach the LLM. TF-IDF similarities are lower-magnitude (typically 0.02–0.25), giving the threshold real discriminative power: 26% of queries (those the model is genuinely uncertain about) are escalated, and the LLM corrects most of them. This confirms that in a confidence-gated cascade, **calibration of the cheap stage matters more than its raw accuracy**.

Table 6: Cheap-stage comparison on full test set (n=5,500). Cascade uses oracle-LLM (upper bound).

R2 model	Params	R2 Acc	ms/q	Cascade	LLM%
TF-IDF (ours)	10K feat.	74.0%	0.3	**86.7%**	26.0
all-MiniLM-L6-v2	22M	73.3%	6.2	75.6%	1.4
all-mpnet-base-v2	109M	**76.2%**	17.1	77.2%	1.3

5 Discussion

When does the cascade pay off? The cascade works because CLINC150's difficulty distribution is bimodal. Most in-domain queries contain obvious keywords ("transfer money," "book flight") that R1 or R2 can handle. The hard tail, mostly meta-intents and OOS, needs the LLM. On a benchmark where every query is equally hard, the savings would be smaller.

Cascade ordering. The R1→R2→R3 ordering follows a cost-ascending principle: keyword matching is essentially free, TF-IDF cosine lookup adds marginal compute, and the LLM API call dominates cost. We do not explore R2→R1→R3 because R2 subsumes R1's discriminative ability: any query resolvable by keyword match is also handled by embedding similarity, but not vice versa. Running the more capable stage first would add cost on the 59% of queries that R1 already resolves.

Latency. The cascade also improves latency. On a Mac Mini (M2), R1 runs in 3.2 ms and R2 in 0.3 ms, both 100–1,500× faster than the ∼500 ms API round-trip for R3. Since 74% of queries stop before reaching the LLM, the median latency drops from hundreds of milliseconds to single digits.

Limitations. (1) We evaluate on a single English-only, single-turn benchmark (CLINC150); generalisation to multilingual or multi-turn routing is untested. (2) The 8-way agent inventory is fixed; cascades on much larger label spaces (50+ agents) would require a different cheap-stage design. (3) The thresholds are static; there is no online adaptation. (4) Haiku 4.5 is itself a cheap model; savings against Claude Sonnet or GPT-4 would be larger in absolute dollars. (5) The cheap-stage comparison uses fixed thresholds tuned for TF-IDF; re-tuning per model would give a fairer comparison but is outside the scope of this workshop paper. (6) R1's keyword lists are hand-crafted for CLINC150's intent names; deploying to a new domain requires rebuilding them, a non-trivial transfer cost that our evaluation does not measure. (7) LLM results are measured on a 400-query stratified subsample per seed; while pooled $n = 1{,}200$ gives tight CIs, the subsample's class balance may not reflect real-world query distributions. (8) Our SetFit baseline uses a compact 17M-parameter backbone; a stronger encoder (e.g., MPNet, 109M) with more training data would raise this number. However, SetFit serves only as a reference point—it is not a cascade component—so the primary R3 vs. R4 comparison is unaffected.

Extension to multi-turn tasks. Single-turn routing is only part of the picture. In multi-turn agent

tasks, the same idea applies at a different granularity: use a cheap model to *decompose* the task and reserve the expensive model for *execution*. We test this on τ-bench (Sierra et al., 2024) in companion work.

6 Conclusion

On CLINC150, the cascade matches the LLM-only router within 0.4 pp while cutting LLM calls by 74%. The practical implication is that an LLM router is only necessary on the hard tail of queries, roughly a quarter of traffic in our setup, and the cutoff can be measured on a team's own data with the code we release.

Acknowledgments

We thank Professor Ramin Sahba for suggesting calibrated routing via Platt scaling (§4.6). An LLM assistant (Claude, Anthropic) was used for proofreading and stylistic editing of the manuscript. All research design, experiments, analysis, and initial drafting were conducted solely by the author.

Reproducibility Statement

Code, tuned thresholds, per-seed trajectories, LLM call logs: `https://github.com/drewOrc/cost-aware-hybrid-router` (MIT). Model: `claude-haiku-4-5-20251001`. Seeds: 42, 43, 44. R2: TF-IDF (scikit-learn). Total API cost: **$0.44**. We verified reproducibility by running the full 3-seed experiment twice: Run 1 vs Run 2 accuracy was 83.75% vs 83.75% (seed 42), 82.50% vs 82.50% (seed 43), 82.00% vs 83.00% (seed 44). All differences are ≤ 0.5 pp, attributable to rate-limited API retries, with all qualitative conclusions unchanged.

References

Gaurav Arora, Shreya Jain, and Srujana Merugu. 2024. Intent Detection in the Age of LLMs. In *Proceedings of EMNLP 2024 Industry Track*, pages 1211–1227.

Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176*.

Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. 2019. An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. In *Proceedings of EMNLP-IJCNLP 2019*, pages 1311–1316.

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP 2019*.

Sierra. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*.

Lewis Tunstall, Nils Reimers, Unso Eun Seo Jo, Luke Bates, Daniel Korat, Moshe Wasserblat, and Oren Pereg. 2022. Efficient Few-Shot Learning Without Prompts. *arXiv preprint arXiv:2209.11055*.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs with Preference Data. In *Proceedings of ICLR 2025*.

Aman Madaan, Pranjal Aggarwal, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. 2024. AutoMix: Automatically Mixing Language Models. *arXiv preprint arXiv:2310.12963*.

Marija Šakota, Maxime Peyrard, and Robert West. 2024. Fly-Swat or Cannon? Cost-Effective Language Model Choice via Meta-Modeling. In *Proceedings of WSDM 2024*, pages 799–807.

Niklas Strauß, Matthias Feurer, and Noor Awad. 2025. Doing More with Less: A Survey on Routing Strategies for Resource Optimisation in LLM-Based Systems. *arXiv preprint arXiv:2502.00409*.

Zhiyuan Li et al. 2026. Dynamic Model Routing and Cascading for Efficient LLM Inference: A Survey. *arXiv preprint arXiv:2603.04445*.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. 2024. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. In *Proceedings of ICLR 2024*.

John C. Platt. 1999. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press.

Jasper Dekoninck, Maximilian Baader, and Martin Vechev. 2025. A Unified Approach to Routing and Cascading for LLMs. In *Proceedings of ICML 2025*.